ARCHITECTURE

[PLATFORM_NAME]

Version 1.0 · Phase 1 — Foundation

System Architecture, Module Map & Infrastructure Topology

> This document is the single source of truth for all architectural decisions. Every engineering decision that touches system structure must be recorded here. Read before any work begins.

---

1. Document Purpose

This document is the single source of truth for all architectural decisions, system design, module boundaries, data flows, and infrastructure topology for [PLATFORM_NAME].

Every engineering decision that touches system structure must be recorded here. This document is read by every new engineer, AI agent, and technical stakeholder before any work begins.

---

2. Executive Summary

[PLATFORM_NAME] is a modular, API-first CMS + SaaS + SEO Tools ecosystem platform designed to scale from a single blog-first website to a multi-tenant enterprise SaaS ecosystem capable of supporting millions of pages, heavy AI workloads, and plugin-based extensibility — without requiring architectural rewrites at any growth stage.

The architecture is built on four guarantees:

1. Portability — runs on any VPS, Docker host, or cloud provider
2. Modularity — every subsystem is independently replaceable
3. Scalability — designed for 10x the current load at every layer
4. Open-source — no paid dependencies required for core functionality

---

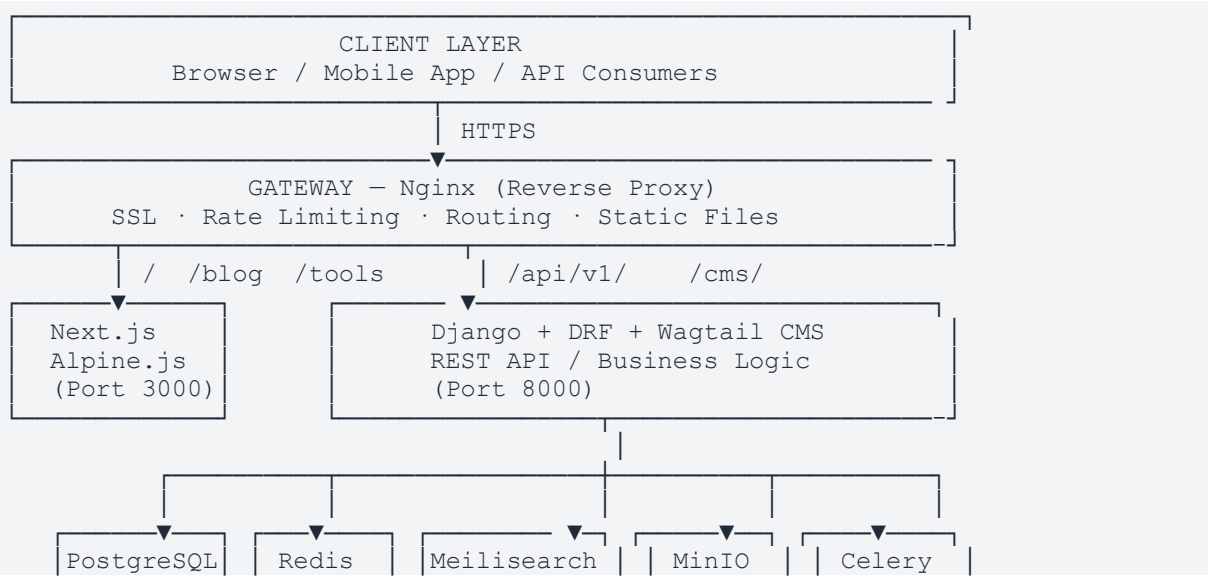
3. Core Architecture Principles

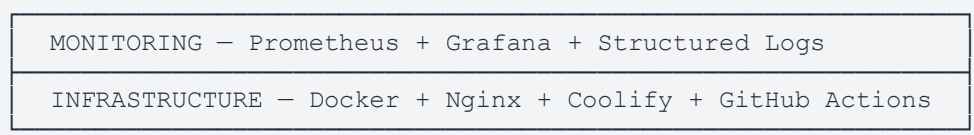
Principle	What It Means
Modular Monolith First	Single deployable unit, modular internals — microservices only when justified
API-First	Every feature exposed via versioned REST API before UI is built
Separation of Concerns	Frontend, backend, CMS, and infra are independently deployable
Open-Source Only	MIT/BSD/Apache 2.0/PostgreSQL licensed stack — no vendor lock-in
Self-Hostable	Full stack runs on a single VPS — no managed services required
Design Token Driven	All frontend visual values are CSS custom properties — no hardcoded hex
Service Layer Pattern	Business logic lives in services.py — not in views, models, or tasks
Async by Default	Heavy operations are always background tasks via Celery
Security by Default	Authentication, RBAC, and input validation on every endpoint
Mobile-First Frontend	UI designed for mobile viewport, progressively enhanced

---

4. System Architecture — Full Layer Map

The platform is organised into 7 distinct horizontal layers, each with a clearly defined role and boundary:





---

5. Layer-by-Layer Architecture

5.1 Gateway Layer — Nginx

Single entry point for all traffic. Responsibilities:

- SSL/TLS termination (Let's Encrypt / self-signed dev)
- Reverse proxy routing to all services
- Static file serving (frontend build artifacts)
- Media file serving (user uploads via MinIO proxy)
- Rate limiting (global + per-route)
- Security headers injection
- Gzip/Brotli compression

Routing Table:

Route	Target
/ /blog/* /tools/* /dashboard/*	Next.js (port 3000)
/api/v1/*	Django (port 8000) — REST API
/cms/* /admin/*	Django (port 8000) — Wagtail + Django admin
/static/*	Nginx — collected Django static files
/media/*	MinIO (port 9000) — uploaded media files

5.2 Frontend Layer — Next.js + Alpine.js

Technology	Use Case
Next.js App Router	App shell, SSR/SSG pages, SEO-critical pages, dashboard

Technology	Use Case
React 18 + TypeScript	Interactive components, data-heavy UI, forms
TailwindCSS	All styling via design token-mapped utility classes
Alpine.js	CMS-rendered HTML pages ONLY — dropdowns, modals, tabs, toggles

Rendering Strategy:

Page Type	Strategy
Marketing pages (/, /blog, /tools)	SSR + ISR (Next.js)
Blog post pages (/blog/[slug])	SSG with ISR revalidation
Tool pages (/tools/[slug])	SSR (personalised)
Dashboard pages (/dashboard/*)	CSR behind auth (protected)
CMS-rendered pages (Wagtail)	Server-rendered HTML + Alpine.js

5.3 Application Layer — Django + DRF + Wagtail

All business logic, content management, and API endpoints.

Strict 4-Layer Architecture:

Layer	Location	Responsibility	Rule
Views / ViewSets	apps/*/views.py	HTTP only — auth, routing, response	Thin — call service and return
Serializers	apps/*/serializers.py	Validate input, format output	Never call services
Services	apps/*/services.py	ALL business logic	Only place decisions are made
Models / ORM	apps/*/models.py	Fields, relationships, __str__	No business logic

5.4 Module Map — Django Apps

App	Phase	Responsibility
core	1	BaseModel, mixins, shared permissions, pagination, exceptions
users	1	Custom user model, profiles, JWT auth, RBAC roles
blog	1	Wagtail blog pages, SEO fields, scheduling, revisions
api	1	DRF versioning root, shared response format, throttling config
tools	2	Tool page models, tool execution API, rate limiting per tier

App	Phase	Responsibility
downloads	2	Product pages, MinIO signed URL delivery, purchase records
resources	2	Curated links, categories, resource hub CMS integration
subscriptions	2	Plans, Stripe billing, usage tracking, API keys
leads	2	Lead capture forms, CRM webhooks, email drip sequences
analytics	3	Event tracking, funnel analysis, reporting
ai	3	LLM clients (Ollama/LiteLLM), prompt chains, AI pipelines, agents

---

6. Database Architecture — PostgreSQL

Primary persistent data store for all application data.

Base Model (inherited by ALL models)

```
class BaseModel(models.Model):
    id         = models.UUIDField(primary_key=True,
                    default=uuid.uuid4, editable=False)
    created_at = models.DateTimeField(auto_now_add=True, db_index=True)
    updated_at = models.DateTimeField(auto_now=True)
    is_active  = models.BooleanField(default=True, db_index=True)
    class Meta:
        abstract = True
        ordering = ['-created_at']
```

Scaling Plan

Stage	Configuration
Phase 1 (< 50k users)	Single PostgreSQL 16 instance
Phase 2 (50k–200k users)	+ PgBouncer for connection pooling
Phase 3 (200k+ users)	+ Read replica for analytics queries
Phase 4 (enterprise)	Tenant schema isolation for multi-tenancy

---

7. Cache Layer — Redis

Caching, session storage, Celery message broker, and rate limiting.

Redis Database Split

Redis DB	Purpose
redis://redis:6379/0	Django cache — query results, page fragments, API responses
redis://redis:6379/1	Celery broker — task queue
redis://redis:6379/2	Celery result backend
redis://redis:6379/3	Rate limiting counters (django-axes, DRF throttling)

Cache Strategy by Layer

Target	TTL	Strategy
Blog post API responses	5 min	Cache-aside, invalidated on publish
Blog listing pages	2 min	Cache-aside
Tool pages (public)	10 min	Cache-aside
User-specific data	No cache	Always live
Sitemap XML	1 hour	Generated async, cached
AI tool results (identical input)	24 hours	Content-addressed cache key

---

8. Search Architecture — Meilisearch

Index	Content	Searchable Fields
blog_posts	All published blog posts	title, body, excerpt, tags, author
tools	All tool pages	name, description, category, tags
downloads	All download products	name, description, category, tags
resources	All resource hub items	title, description, url, category

> Phase 3: Meilisearch vector search (v1.6+) + Ollama embeddings (nomic-embed-text) + Qdrant for cross-model portability. Hybrid search: keyword + vector score combined.

---

9. Storage Architecture — MinIO

All file storage — media uploads, user files, digital downloads. S3-compatible (boto3 / django-storages).

Bucket Structure

```
platform-media/
├── blog/           # Blog post featured images
├── avatars/        # User profile photos
├── cms/            # Wagtail-managed media
└── temp/           # Temporary uploads (auto-cleared 24h)

platform-downloads/
├── products/       # Digital download files (private)
└── previews/       # Public preview files

platform-assets/
└── static/         # Collected Django static files (prod)
```

Bucket	Access Control
platform-media	Public read (CDN-cacheable)
platform-downloads	Private — signed URL, 15-minute expiry after purchase
platform-assets	Public read

---

10. Background Jobs — Celery

Queue	Priority	Workers	Use Case
critical	Highest	2	Email delivery, payment webhooks
default	Normal	4	Search indexing, notifications, file processing
ai	Normal	2	AI generation tasks (can be slow)
maintenance	Lowest	1	Cleanup, nightly jobs, re-indexing

Celery Beat Scheduled Tasks

Schedule	Task
Every 5 min	Sync unpublished content from Meilisearch
Every 15 min	Process pending email queue
Every 1 hour	Aggregate analytics events
Every 6 hours	Generate sitemap XML
Every 24 hours	Full Meilisearch re-index + clean temp MinIO bucket
Every 7 days	Generate weekly analytics report

---

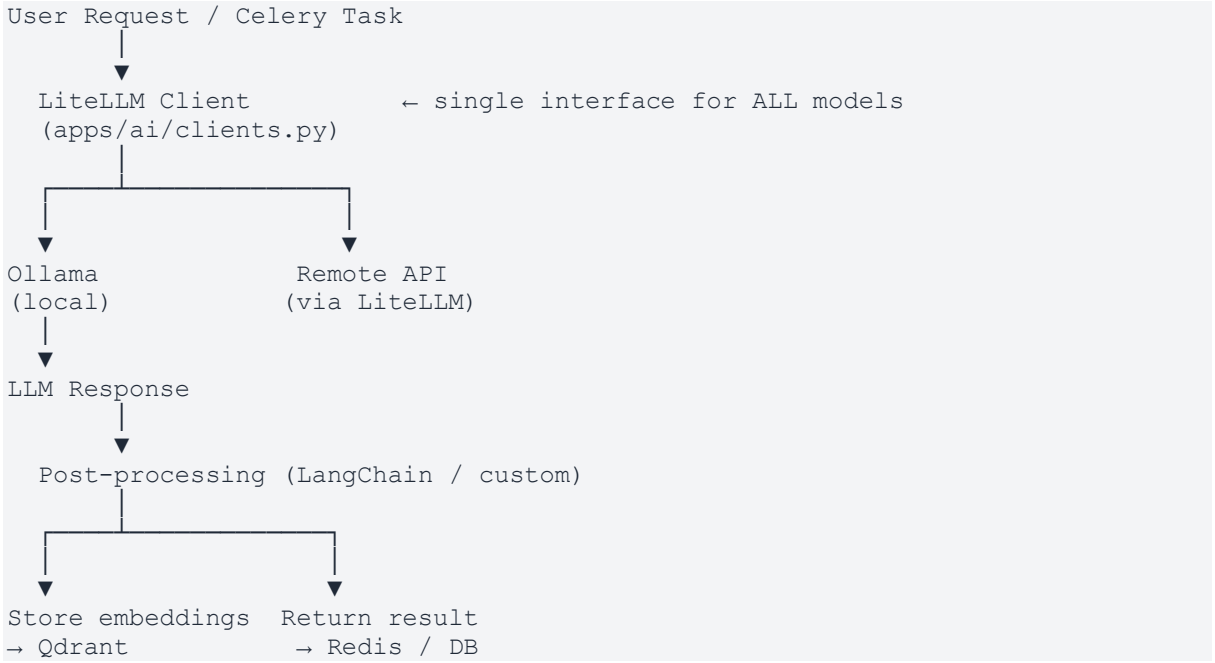
11. AI Layer (Phase 3) — Ollama + LiteLLM + Qdrant

All AI inference, embedding generation, and vector storage. Added in Phase 3.

AI Stack Priority Order

5. Ollama — local LLM inference (default — free, self-hosted)
6. LocalAI — OpenAI-compatible local inference (fallback)
7. LiteLLM — multi-provider abstraction (all AI calls go through this)
8. LangChain / Haystack — agent orchestration, prompt chains
9. Qdrant — vector storage for embeddings
10. Proprietary APIs (OpenAI, Anthropic) — optional only, via LiteLLM

AI Request Flow



---

12. Authentication & Security Architecture

JWT Authentication Flow

11. User submits login credentials
12. Django validates credentials
13. Django issues: access_token (15 min TTL) in JSON body + refresh_token (7 day TTL) in httpOnly cookie
14. Frontend stores access_token in Zustand memory — NOT localStorage

15. API requests send: Authorization: Bearer {access_token}
16. On 401 → Axios interceptor auto-calls /api/v1/auth/token/refresh/
17. Django validates refresh_token cookie → issues new access_token
18. Logout: refresh_token blacklisted + cookie cleared

RBAC Roles

Role	Access Level
ADMIN	Full platform access
EDITOR	CMS content management
MODERATOR	Content review, user management
SUBSCRIBER	Paid plan access to tools/downloads
FREE_USER	Basic access, rate-limited tools
API_USER	API key based access, no UI login

Security Layers

Layer	Mechanism	Scope
1 — Global rate limit	Nginx	100 req/min per IP
2 — Endpoint throttle	DRF throttle classes	Varies by endpoint type
3 — Brute force	django-axes	Lock after 5 failed logins
4 — JWT validation	simplejwt	Every authenticated request
5 — RBAC	Custom permission class	Every view
6 — Object level	Ownership verification	Resource-specific

---

13. API Architecture

URL Conventions

```
GET    /api/v1/blog/posts/              → list (paginated)
POST   /api/v1/blog/posts/              → create
GET    /api/v1/blog/posts/{id}/         → retrieve
PATCH  /api/v1/blog/posts/{id}/         → partial update
DELETE /api/v1/blog/posts/{id}/         → delete
POST   /api/v1/blog/posts/{id}/publish/ → custom action
```

Standard Response Envelope

```
// Success
{ "success": true, "data": {},
  "message": null, "errors": null,
  "meta": { "page": 1, "per_page": 20, "total": 100 } }
```

```
// Error
{ "success": false, "data": null,
  "message": "Human-readable description",
  "errors": { "field": ["detail"] },
  "code": "ERROR_CODE" }
```

Rate Limiting Plan

Endpoint Type	Free User	Subscriber	API Key
Blog read	1000/hour	Unlimited	5000/hour
Tool execution	10/hour	100/hour	1000/hour
AI tool	3/hour	50/hour	200/hour
Auth endpoints	5/min	5/min	5/min
Download	5/hour	Unlimited	50/hour

---

14. Infrastructure Architecture

Docker Services

Service	Purpose
nginx	Reverse proxy — ports 80, 443
backend	Django + Gunicorn — port 8000
frontend	Next.js — port 3000
db	PostgreSQL 16 — port 5432
redis	Redis 7 — port 6379
celery_worker	Celery workers — no exposed port
celery_beat	Celery scheduler — no exposed port
meilisearch	Search — port 7700
minio	Object storage — ports 9000, 9001
prometheus	Metrics collection — port 9090
grafana	Monitoring dashboards — port 3001
ollama (Phase 3)	LLM inference — port 11434

Service	Purpose
qdrant (Phase 3)	Vector DB — port 6333

> All services communicate by Docker service name (DNS). Only Nginx is exposed externally (ports 80, 443). All other ports are internal only.

---

15. CI/CD Architecture

```
Developer pushes to feature branch
         │
         ▼
   GitHub Actions (CI)
   1. Lint (ruff, eslint)
   2. Type check (mypy, tsc --noEmit)
   3. Unit tests
   4. Integration tests
   5. Security scan (bandit, npm audit)
   6. Build check
         │ passes
         ▼
   PR Review Required
         │ approved + merged to main
         ▼
   GitHub Actions (Deploy)
   1. Build Docker image
   2. Push to registry
   3. Notify Coolify webhook
   4. Coolify rolling restart
   5. Run DB migrations
   6. Health check
```

Branch Strategy

Branch	Purpose
main	Production (protected — requires PR + review)
develop	Staging (integration branch)
feature/*	Feature branches (branch from develop)
hotfix/*	Emergency fixes (branch from main)

---

16. Monitoring & Observability

Metrics Collected

System	Metrics
System	CPU, memory, disk I/O per container
Django	Request count, latency, error rate, DB query time
Celery	Queue depth, task success/failure rate, task duration
PostgreSQL	Connections, query time, table sizes, vacuum stats
Redis	Memory, hit rate, evictions, connected clients
Meilisearch	Index size, search latency, indexing rate
Next.js	Core Web Vitals (LCP, CLS, INP) via custom endpoint

Alerting Rules

Priority	Trigger	Action
P0 — Page immediately	Service down, DB unreachable, disk > 95%	Page on-call immediately
P1 — Alert within 1h	Error rate > 5%, latency p99 > 2s, queue > 1000	Alert team
P2 — Alert within 24h	Disk > 80%, cache hit rate < 70%	Create ticket

---

17. Scalability Architecture

Stage	VPS Spec	Handles	Est. Cost/mo
Phase 1	4-8 CPU, 8-16GB RAM, 100GB SSD	~10k DAU, ~100k pageviews	$20–40
Phase 2 + PgBouncer	8-16 CPU, 32GB RAM, 500GB SSD	~50k DAU, ~500k pageviews	$80–120
Phase 3 + CDN + Replicas	2–4 app servers, LB, read replica	~500k DAU, ~5M pageviews	$300–600
Phase 4 + k8s	Kubernetes + multi-region	Millions of users	Scales with usage

---

18. Phase Evolution Map

Phase	New Apps	New Services
Phase 1 — Foundation	core, users, blog, api	All core Docker services, Nginx, GitHub Actions
Phase 2 — Commerce	tools, downloads, resources, subscriptions, leads	Stripe webhooks, MinIO signed URLs, API key system
Phase 3 — AI	ai, analytics	Ollama, Qdrant added to docker-compose
Phase 4 — Enterprise	tenants, plugins, enterprise	Kubernetes migration (optional), multi-region

---

19. Key Technical Decisions & Rationale

Decision	Choice	Trade-off
Architecture style	Modular monolith	No distributed complexity early / harder to scale components independently
Frontend	Next.js App Router	SSR + server components / more complex than CRA
CMS	Wagtail	Django-native, customisable / steeper than headless CMS
CMS interactivity	Alpine.js (not React)	Zero build step in templates / limited to simpler interactions
Primary keys	UUID	No enumeration attacks / slightly larger index
Auth	JWT + httpOnly cookie	Refresh token not JS-accessible / more complex than sessions
Business logic	services.py	Testable, reusable / extra file per app
Object storage	MinIO	Self-hosted S3-compatible / must manage storage infra
Search	Meilisearch	Fast, open-source MIT / less powerful than Elasticsearch
AI inference	Ollama first	Free, private, self-hosted / limited models vs proprietary APIs
Deployment	Coolify	Open-source PaaS / additional service to maintain
Design system	CSS tokens only	Brand change = 1 file / must discipline entire team to use tokens

---

20. Risk Assessment

Risk	Likelihood	Impact	Mitigation
Redis single point of failure	Medium	High	Redis Sentinel in Phase 2
MinIO storage exhaustion	Low	High	Storage alerts at 70%, auto-archive cold files
Celery task queue backup	Medium	Medium	Queue depth alerts, auto-scaling workers
JWT refresh token leak	Low	High	httpOnly + SameSite=Strict + HTTPS only
Database connection exhaustion	Medium	High	PgBouncer in Phase 2, CONN_MAX_AGE now
Meilisearch index drift	Low	Low	Nightly full re-index + publish hooks
Ollama GPU memory exhaustion	Medium	Medium	Request queue + timeout + fallback to remote API
Dependency license change	Low	High	Monitor licenses, pin versions, evaluate on upgrade

---

21. Open Architecture Questions

Pending decisions that affect architecture when confirmed:

#	Question	Needed By
1	Brand name / domain → Nginx config, CORS, email	Before Phase 1 start
2	Analytics (PostHog / Umami / Plausible) → analytics app	Phase 2 start
3	Email provider (Resend / Postal self-hosted) → leads app	Phase 2 start
4	Payment region / currency → Stripe config	Phase 2 start
5	Icon library (Heroicons / Lucide) → all UI components	Phase 1 week 2
6	Brand color palette → globals.css tokens	Phase 1 (any time)
7	CDN provider (Cloudflare / Bunny) → Nginx + media URLs	Phase 3 start

---

This document is the architectural foundation of [PLATFORM_NAME].

All changes to system structure must be reflected here.